

V.B Testing

Action Summary

Management should maintain testing policies, standards, procedures, as well as other process controls that are effectively used to mitigate risks in internally and externally developed systems and components.

Examiners should review the following:

- Testing policies, standards, and procedures, as well as other process controls, to evaluate how well entity personnel use testing to confirm that systems and components meet the entity's requirements.
- Testing scope documentation, including for application interoperability and vulnerability discovery.
- Documentation regarding controls over the use of production data in testing.
- Documentation of the type of testing, testing results, corrective actions, and testing completion.
- Documentation of testing for any new controls added to systems and components to avoid conflicts (e.g., integration and interface, security) resulting in failed business processes.
- Any updates to manuals and training plans after testing.

Testing is essential to the development process to promote confidentiality, integrity, availability, and resilience. Whether developed internally or by a third party, appropriate personnel should test systems and components to identify and correct defects before deployment, including security-related defects.

Testing should be sufficient to confirm that systems and components meet the entity's architectural, functional, information security, and legal/regulatory requirements. To help mitigate the risk that an integration point contains an undiscovered defect, entities test the system or component being developed, including how well that system or component interoperates with other systems and components. This scope is commonly achieved through a combination of code-level, system-level, and code vulnerability testing.

Starting test plans no later than the development and acquisition phase helps teams test more comprehensively than if they wait to create test plans later in the SDLC. Additionally, the level of detail in the test plan and script should match the level of risk the system or component presents to the examined entity's business. The more detailed test plans and scripts are, the higher the likelihood that testers will identify defects before system or component implementation.

Using production data in testing may provide assurance that the system or component functions as expected; however, there are significant risks (e.g., unauthorized access to or modification of sensitive entity or customer information). Management should determine the need for using

production data in testing and employ appropriate controls if its use is deemed necessary. When possible, testers should use simulated synthetic data or sanitized¹⁸⁴ production data during system or component development and testing¹⁸⁵ to protect sensitive customer and entity data. Data used for testing should not include sensitive customer information unless the data are sanitized¹⁸⁶ before testing, and this should be addressed in policy. Due to the risks involved with actual customer data, when sanitizing data is not feasible, management should document an exception to board-approved policy for its use. Implementation and maintenance of controls similar to those used in the production environment should be used to appropriately protect the data for compliance with legal and regulatory requirements. The Information Security Standards require safeguarding of sensitive customer information regardless of the environment (e.g., testing, QA, or production) where the information resides.¹⁸⁷ For more information, refer to the *FFIEC IT Handbook's* "[Architecture, Infrastructure, and Operations](#)" and "[Information Security](#)" booklets.

The types of testing¹⁸⁸ performed should be appropriate for the development activity's inherent risk. Effective management has a methodical process to define and conduct testing necessary to demonstrate the effectiveness of a developed system or component. Typically, testing is performed in stages, and knowledgeable testing specialists and relevant stakeholders should be involved in testing. Regardless of the process, various tests may be used individually or in combination at different points in the development and testing phases. During system or component development, developers perform code reviews, which typically occur as part of the testing process. Appropriate personnel can perform testing manually, with automated tools, or a combination of both.

¹⁸⁴ Data sanitization disguises (i.e., anonymizes) sensitive information in test and development databases by overwriting the information with realistic looking but false data of a similar type. Some forms of sanitization include NULLing out, masking data, substitution, shuffling records, and encryption. For more information, refer to NIST SP 800-160, vol. 1, rev. 1, [Engineering Trustworthy Secure Systems](#).

¹⁸⁵ Refer to NIST SP 800-160, vol. 1, rev. 1, [Engineering Trustworthy Secure Systems](#).

¹⁸⁶ Data sanitization can help minimize risks related to unauthorized access of that data. Unsanitized data passed to other systems or subsystems (e.g., command shells, relational databases, and commercial off-the-shelf [COTS] components) may be vulnerable. Attackers may be able to exploit unused functionality to access, manipulate, or exfiltrate the data causing legal, reputation, and operational risk to the entity. Additionally, that data may provide information to attackers, allowing for unauthorized lateral movement throughout the network, systems, and components.

¹⁸⁷ Refer to [GLBA](#), further implemented by FFIEC members as follows: FDIC: 12 CFR 364, appendix B, "[Interagency Guidelines Establishing Information Security Standards](#)," and 12 CFR 364, supplement A to appendix B, "[Interagency Guidance on Response Programs for Unauthorized Access to Customer Information and Customer Notice](#)"; FRB: Regulation H, 12 CFR 208, appendix D-2, "[Interagency Guidelines Establishing Information Security Standards](#)," and Regulation Y, 12 CFR 225, appendix F, "[Interagency Guidelines Establishing Information Security Standards](#)"; NCUA: 12 CFR 748, appendix A, "[Guidelines for Safeguarding Member Information](#)"; OCC: 12 CFR 30, appendix B, "[Interagency Guidelines Establishing Information Security Standards](#)." Additionally, refer to FTC 16 CFR 314, "[Standards for Safeguarding Customer Information](#)," for similar provisions for service providers.

¹⁸⁸ For purposes of this section, NIST IR 8397, [Guidelines on Minimum Standards for Developer Verification of Software](#), defines "testing" as "any technique or procedure performed on the software itself to gain assurance that the software will perform as desired, has the necessary properties, and has no important vulnerabilities."

While the testing types, such as dynamic testing, may be performed during multiple phases, commonly performed testing in the development and acquisition phase includes the following:

- **Static analysis:** Personnel detect software vulnerabilities by examining the application source code and binary code and attempting to analyze and identify all possible behaviors that might arise at runtime.¹⁸⁹ An example of static analysis is software application security testing, which analyzes the quality of the code.¹⁹⁰
- **Unit testing:**¹⁹¹ Developers perform unit tests to assess the functionality of small modules of code.
- **Security testing:** Developmental testing helps ensure that the technical and security features and functions of the system perform as intended.¹⁹² This may include initial penetration testing of the code.
- **Dynamic analysis:** Personnel use testing that operates by executing a program using a set of input use cases and analyzing the program's runtime behavior. Testing may be extensive (i.e., time-consuming) depending on the number of scenarios analyzed. An example of dynamic, execution-based scans is dynamic application security testing, which is intended to simulate real-world attacks.¹⁹³

The development team should document the results of testing performed. Documentation should include the type of testing performed, scope of the testing, test results, further action based on results of the test (e.g., error correction, patch, or retest), and whether the testing is considered complete. Appropriate personnel can provide a report communicating the test results to management as appropriate.

Commonly performed testing in the implementation and assessment phase includes the following:

- **System testing:** Testing is performed on a complete system to evaluate its compliance with specified requirements.
- **Additional security testing:** If new controls are added to the system, additional acceptance tests of those new controls should be performed to help ensure that new controls meet security specifications and do not conflict with or invalidate existing controls or functionality.
- **Integration testing:** Personnel perform an orderly progression of testing in which software elements, hardware elements, or both are combined and tested to evaluate their interactions,

¹⁸⁹ Refer to NIST 800-163, rev. 1, [Vetting the Security of Mobile Applications](#).

¹⁹⁰ For more information, refer to [OWASP Vulnerability Management Guide \(OVMG\)](#).

¹⁹¹ Information Systems Audit and Control Association (ISACA) Glossary defines “[unit testing](#)” as “a testing technique that is used to test program logic in a particular program or module. The purpose of the test is to ensure that the internal operation of the program performs according to specification. It uses a set of test cases that focus on the control structure of the procedural design.”

¹⁹² Refer to NIST ITL Bulletin, “[The System Development Life Cycle](#).”

¹⁹³ For more information, refer to [OWASP Vulnerability Management Guide \(OVMG\)](#).

until the entire system has been integrated.¹⁹⁴ A primary goal of integration testing is to ensure that systems and components function appropriately together. This is especially difficult when software uses components developed by different vendors, in different languages, and the implementation sources are not all available.¹⁹⁵

- **UAT:** Personnel perform testing that involves taking use cases or procedures for how the system was designed to perform and ensuring that someone who follows the procedure gets the intended result; however, it does not necessarily demonstrate how well the system supports users in performing those functions.¹⁹⁶ Therefore, UAT is not used to determine usability. UAT generally occurs toward the end of development testing activities. UAT may be useful for validating procedures and user training on the operation of new or significantly changed systems and components.
- **Regression testing:** Personnel execute testing that involves the performance of the same tests many times to determine whether to return to the prior state. A retest of a system or component helps to assess functionality after programmers make code changes following previous tests. Regression testing generally includes systems that may not have changed but may be affected by a new or changed system or component.
- **Stress testing:** For purposes of this discussion, this refers to testing performed to determine the limitations of a system or component. IT personnel periodically test to determine whether the entity's system or component can handle the anticipated maximum volume or duration of activity while maintaining required functionality. This may be performed using automated testing tools that generate simulated scenarios.

Regardless of the testing methods used, management should implement practices to document, report, and address identified issues, including security-related issues, in a timely manner. Tracking corrections and modifications resulting from testing facilitates the completeness of the overall program documentation. During the implementation and assessment phase, effective management reviews and finalizes all supporting documentation, such as user, operator, and maintenance manuals as well as any conversion, implementation, and training plans associated with a new release or significant update.

V.C DevOps and DevSecOps

DevOps builds on agile principles and is a software engineering culture and set of practices designed to unify software development and operations. DevSecOps takes DevOps one step further by integrating security ("Sec" stands for "security"). The following subsections of this booklet provide more information on DevOps and DevSecOps.

¹⁹⁴ Refer to [ISACA Glossary](#).

¹⁹⁵ For more information, refer to Leonard J. Gallagher and Jeff Offutt, [Test Sequence Generation for Integration Testing of Component Software](#).

¹⁹⁶ Refer to NIST IR 7741, [NIST Guide to the Processes Approach for Improving the Usability of Electronic Health Records](#).

V.C.1 DevOps

DevOps is a “set of practices for automating the processes between software development and information technology operations teams so that they can build, test, and release software faster and more reliably. The goal is to shorten the SDLC and improve reliability while delivering features, fixes, and updates frequently in close alignment with business objectives.”¹⁹⁷ DevOps is used to promote coordination and communication between development and operations teams (e.g., developers, testers, or operations personnel). For example, a group of developers can be embedded with the operations team to acquire domain-specific expertise, allowing them to better understand applicable business processes in a given area of the entity’s systems. The operations team benefits from having developers readily accessible to help operations personnel better understand the entity’s developed applications and IT infrastructure.

Using DevOps practices, personnel can automate tasks that were previously performed manually. For example, a DevOps team can automate configuration management processes (e.g., firewall rules, server hardening, and mobile device management). DevOps practices often include containerization, which allows applications to be implemented as microservices. For more information on containers and microservices, refer to the “[Common Development, Acquisition, and Maintenance Risk Topics](#)” section of this booklet.

Automation and orchestration give the DevOps approach several benefits including shorter development cycles, increased deployment frequency, faster time to market, and more stable operating environments. In the past, entities may have had changes that took place on a weekly basis; however, in a DevOps environment, the same number and type of changes can be made daily or even hourly. However, automation of previously manual activities, combined with frequent and rapid release cycles, may result in perpetuation of errors or invalid code. Management should assess the risks involved in using a DevOps approach and implement appropriate controls. Potential risks include the following:

- Misconfigured process automation (e.g., quality and release management).
- Unclear governance practices, such as inadequate development standards and procedures and a lack of validation of effectiveness.
- Inappropriate metrics or tracking measures.
- Unauthorized code access and modification via unsecured code repositories (e.g., GitHub).
- Lack of cyber hygiene (e.g., inappropriate configurations or unscanned and vulnerable systems are released).
- Developers being able to use uncontrolled and insecure system images.
- Inadequate controls over code implemented in production (e.g., bypassing the entity’s coding standards).

V.C.2 DevSecOps

DevSecOps is the evolutionary progression from DevOps that integrates security objectives and controls (e.g., metrics and compliance reports) into the development process. The main

¹⁹⁷ Refer to NIST SP 1800-16, [Securing Web Transactions: TLS Server Certificate Management](#).

characteristics of this practice are to automate, monitor, and apply security at all phases of the SDLC. Automation of the workflows and integrated security testing is a key attribute of DevSecOps. Additionally, automation of testing (e.g., unit testing, code analysis, and image scanning) provides timely feedback to DevSecOps teams while not obstructing productivity. Cybersecurity and design have the same priority as they would in traditional system and component development. Security resources are integrated at the outset of the development process. DevSecOps is a key approach used in cloud development environments. Cloud-native applications require updates and deployment techniques to be flexible and secure for business reasons, as well as resilient to respond to cybersecurity events.¹⁹⁸

Coding services and tools can be used to automate and implement security faster in the development process, allowing for quicker implementation. NIST notes that DevSecOps can be used with the following types of coding and coding services:¹⁹⁹

- **Application code:** Code used for one or more business functions, made up of code describing the business transactions and any database access.
- **Application services code (e.g., service mesh code):** Provides various services for the application, such as service discovery, establishing network routes, network resiliency services (e.g., load balancing, retries), and security services (e.g., enforcing authentication and authorization based on policies).
- **Infrastructure as code:** Expresses the computing, networking, and storage resources needed to run the application in the form of declarative code.
- **Policy as code:** Contains declarative code for generating the rules and configuration parameters for realizing security objectives through security controls (e.g., authentication and authorization) during runtime.
- **Observability as code:** Code that triggers automation of software related to logging all transactions, tracing communication pathways involved in executing application requests, and monitoring applications during runtime.

As infrastructure and development activities have moved to a cloud-based environment, the use of the term “as code” is often added to the type of technique. Further, “when using ‘as code’ techniques, the code that is written (e.g., for provisioning a resource) is managed like application source code. This implies that it is versioned, documented, and has access controls defined similarly to what is done for an application source code repository.”²⁰⁰ In addition, some on-premises hardware components may be replaced and deployed as software in cloud-based environments.

¹⁹⁸ Refer to NIST SP 800-204C, [Implementation of DevSecOps for a Microservices-Based Application With Service Mesh](#).

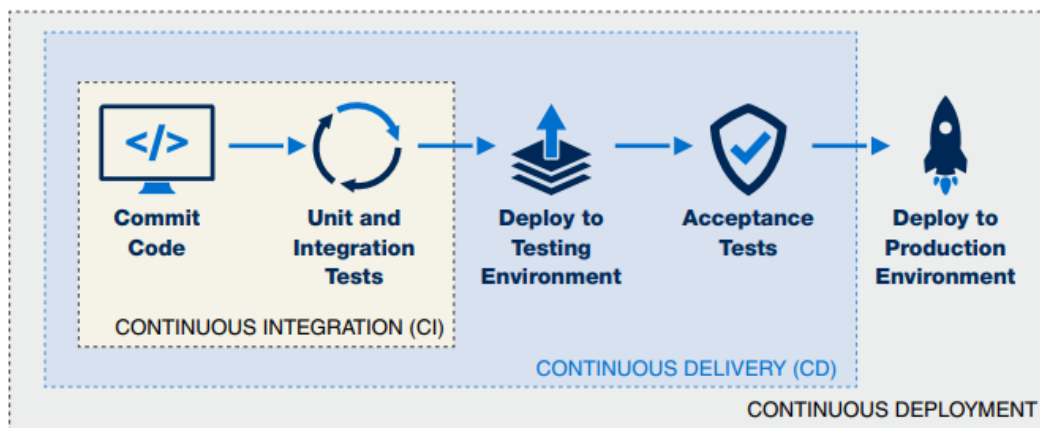
¹⁹⁹ [Ibid.](#)

²⁰⁰ [Ibid.](#)

DevSecOps makes use of the continuous integration (CI)²⁰¹ and continuous delivery (CD)²⁰² or continuous deployment²⁰³ pipeline (see figure 6) during the SDLC, depending on whether the entity uses automation to move code into production. The U.S. Department of Defense (DOD) defines this CI/CD pipeline as “the set of tools and the associated process workflows to achieve CI/CD or continuous deployment with build, test, security, and release delivery activities, which are steered by a CI/CD orchestrator and automated as much as practice allows.”²⁰⁴

Developers digitally sign code in the CI/CD pipeline process. This helps with determining code integrity. For example, downstream evaluations can compare the expected code author’s name to the name of the person who last changed the code and take appropriate action if they do not match.

Figure 6: CI/CD Pipeline Process Workflow



During CI (as illustrated above), developers frequently merge code changes into a central repository where automated builds and tests run.²⁰⁵ CI activities include committing code (i.e., establishing

²⁰¹ Continuous integration (CI) involves developers frequently merging code changes into a central repository where automated builds and tests run. Build is the process of converting the source code to executable code for the platform on which it is intended to run. When developers use CI to deploy new software, the developer’s changes are validated by creating a build and running automated tests against the build. This process avoids the integration challenges that can happen when waiting for release day to merge changes into the release branch. For more information, refer to NIST SP 800-204C, [Implementation of DevSecOps for a Microservices-Based Application With Service Mesh](#).

²⁰² NIST defines “continuous delivery” as “the stage after CI when code changes are deployed to a testing or staging environment after the build stage.” For more information, refer to NIST SP 800-204C, [Implementation of DevSecOps for a Microservices-Based Application With Service Mesh](#).

²⁰³ NIST defines “continuous deployment” as “similar to continuous delivery except that the releases happen automatically, and changes to code are available to customers immediately after they are made.” For more information, refer to NIST SP 800-204C, [Implementation of DevSecOps for a Microservices-Based Application With Service Mesh](#).

²⁰⁴ Refer to DOD’s [DoD Enterprise DevSecOps Reference Design: Version 1.0](#).

²⁰⁵ For a definition of CI, refer to NIST SP 800-204C, [Implementation of DevSecOps for a Microservices-Based Application With Service Mesh](#).

version control, in which a snapshot is taken of the current code changes and saved to a repository), developers packaging up code (i.e., bundling of related code) or revisions to code, and automated initial code testing (such as unit and integration testing).

Management often uses CI processes to put the entity's apps into containers directly in the build process itself. Also, during CI, code is uploaded into a central repository where automated tools validate and initially test the code. The purpose is to avoid inconsistency from the very beginning of an app's life cycle to help ensure consistency in the app's runtime environment (i.e., the environment in which a program is executed). Many organizations may see developers build in one environment, test in another, and deploy in a third, so having consistency in assessment and enforcement across these environments is key.²⁰⁶

Once the code is initially validated and tested, CD automatically deploys code to the formal testing environment for acceptance testing. Within the CD segment illustrated above, code is deployed to production manually, using scripts to assist in the configuration. In continuous deployment, rather than manually, deployment occurs when the entity automates the deployment process, and code is available to users immediately.

NIST documents the following resources and tools used in the CI/CD pipeline process:²⁰⁷

- **Pipeline software.**
 - CI software: Pulls code from a code repository for building the software, facilitates initial testing tools, and saves tested artifacts to an image registry (i.e., repository for copies of code to maintain coding integrity).
 - CD or continuous deployment software: Pulls artifacts from the repository and deploys the package.
- **Development software.**
 - Build tools: Used for constructing software (e.g., for integrated development environments).
 - Testing tools (e.g., software application security testing, dynamic application security testing, or software composition analysis).
- **Repositories.**
 - Source code repositories (e.g., GitHub).
 - Container image repositories or registries.
- **Observability or monitoring tools.**
 - Logging and log aggregation tools.
 - Tools that generate metrics.
 - Tracing tools (e.g., sequence of application calls).

²⁰⁶ Refer to NIST SP 800-190, [Application Container Security Guide](#).

²⁰⁷ For more information, refer to NIST 800-204C, [Implementation of DevSecOps for a Microservices-Based Application With Service Mesh](#).

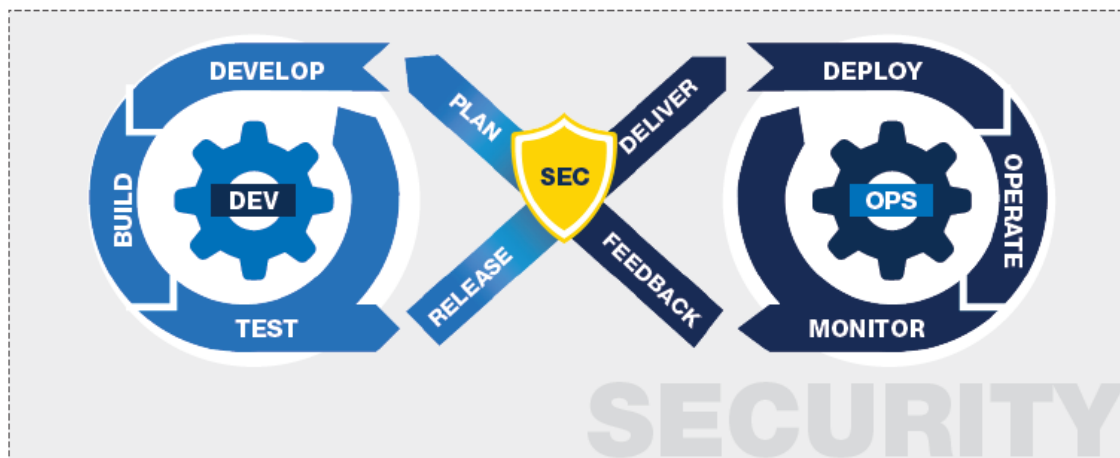
- Visualization tools (e.g., used to combine data from all development activities to generate dashboard/alerts).

NIST recommends the following controls to secure the CI/CD pipeline process:²⁰⁸

- Harden servers hosting code and artifact repositories.
- Secure credentials (e.g., authorization tokens) used for accessing repositories.
- Implement controls on who can check in and check out artifacts in container image registries. These registries store artifacts produced by the CI pipeline and serve as bridges between the CI and CD pipelines.
- Log all code and build update activities.
- Send build reports to developers and stop further pipeline tasks when a build or test fails in the CI pipeline. Code repositories should be configured to automatically block all subsequent pull requests from the continuous deployment stage of the pipeline until issues are resolved.
- Send build reports to the security team and stop further pipeline tasks when an audit fails.
- Ensure that developers can only access the application code.
- Digitally sign (preferably multiparty digital signing) the release artifact during each required CI/CD stage during the build and release process.
- Verify that all required digital signatures are present during production release to ensure that no one bypasses the pipeline.

Figure 7 provides an overview of the CI/CD pipeline. Automated security testing is performed throughout development phases and throughout production.

Figure 7: CI/CD Pipeline Architecture²⁰⁹



²⁰⁸ Refer to NIST SP 800-204C, [Implementation of DevSecOps for a Microservices-Based Application With Service Mesh](#).

²⁰⁹ Refer to DOD's [DoD Enterprise DevSecOps Reference Design: Version 1.0](#).

V.D Functional Development Types

Functional development types include hardware, software, front end, and backend. Examples of hardware developed for the financial sector are point-of-sale terminals, mobile card readers, credit cards, and managed security devices. Examples of software developed for the financial sector are mobile banking apps, online banking systems, loan estimators, core banking systems, transaction monitoring systems, and imaging systems. Development of either hardware or software may include a combination of backend and front-end development types. Backend refers to the parts of systems that are not seen by the end user. Front end refers to the parts of systems that the end user sees and uses. Understanding that there are different functional development types is important for development teams as each development type may have unique procedures (e.g., steps and timing), include various reporting lines (e.g., internal and external stakeholders, including management), and address different risks (e.g., physical and virtual) depending on the project.

Functional development types can include the following:

- **Hardware development:** Hardware development includes integrating hardware with software to provide a user experience that supports the business use cases. This includes designing and building the physical aspects of a hardware component. Hardware may include servers, circuit boards, credit cards, ATMs, point-of-sale terminals, and mobile reader devices. The hardware developer designs the system, selects components, builds prototypes, and coordinates testing. The developer generally has in-depth knowledge of software development because developing hardware requires using complex design tools and programming skills. Hardware development includes incorporating firmware and producing circuitry concepts and circuit diagrams, which are then put into operation. The developer designs circuit board layouts and programs microchips and microcontrollers. They develop and simulate digital signal and image processing. They are responsible for maintenance and testing as well as preparing manufacturing documents and product documentation. Hardware developers are sometimes referred to as electronics developers or hardware designers.
- **Front-end (or client-side) development:** Front-end²¹⁰ development can be facilitated through hardware or software-based solutions. A front-end developer works toward providing an effective end-user experience. This includes anything (e.g., text and layout, touchscreen, and icons) the user interacts with on their devices (e.g., PCs, tablets, mobile phones, and ATMs). Developers combine design, technology, and programming to create user interfaces that provide clear and understandable products with end-user appeal. Another front-end consideration is the importance of integrating the business line product rules (e.g., rules for negative interest rate changes or loan-to-value limitations) into the user interface. Front-end development may include web development, desktop development, mobile development, and graphics development.

²¹⁰ Merriam-Webster defines “[front end](#)” as “a software interface (such as a graphical user interface) designed to enable user-friendly interaction with a computer.”